■ SRE RAG Agent v4.0 — Guia de Execução Local

Passo a Passo (Step by Step) para Inicialização, Operação e Validação do Laboratório

Este documento reúne as 3 formas homologadas de subir e operar o ambiente do **SRE Incident Simulation Lab v4.0** em ambiente local (WSL2 / Ubuntu ou Linux). Escolha a opção mais adequada ao seu objetivo de estudo ou teste.

■ **Novidades & Arquitetura da Versão v4.0:**
- **Agente SRE com Ciclo OODA:** Raciocínio closed-loop (Observe, Orient, Decide, Act) com Tool Calling e streaming SSE.
- **Busca Híbrida 3-Stage:** pgvector (HNSW) + BM25Okapi + Fusão RRF (k=60) + Re-ranking Neural FlashRank.
- **Chaos Engineering Real:** Injeção no kernel (cgroups v2, CFS throttling, tc netem) e Dead Man's Switch de 60s.
- **OpenTelemetry & Jaeger:** Spans distribuídos (FastAPI, DB, Redis) com trace_id e span_id no structlog.
- **DevSecOps PSS Restricted:** readOnlyRootFilesystem, usuário não-root, NetworkPolicies e 0 CVEs Trivy.

Opção 1: Modo Desenvolvimento Local Rápido [MAIS RÁPIDO]

Ideal para desenvolvimento, testes de UI, simulação de incidentes, War Room e diagnóstico com o Agente SRE sem Docker.

Passo 1: Navegue até o diretório raiz do projeto:

```
cd /home/magno/sre_rag_agent
```

Passo 2: Crie e ative o ambiente virtual Python 3.12 (caso ainda não exista):

```
python3 -m venv app/.venv
source app/.venv/bin/activate
```

Passo 3: Instale as dependências (com PyTorch CPU otimizado, sem CUDA bloat):

```
pip install --upgrade pip
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r app/requirements.txt
```

Passo 4: Inicie o servidor ASGI (Uvicorn) na porta 8080:

```
cd app
../app/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8080 --reload
# Ou execute diretamente na raiz: make dev
```

Passo 5: Acesse a interface e endpoints no navegador:

Dashboard Principal & Quiz	http://localhost:8080/dashboard	Simulador de incidentes, vitrine e runbooks em 5 fases
War Room & Agente OODA	http://localhost:8080/dashboard#warroom	Terminal seguro e painel de diagnóstico do Agente SRE v4.0
Conceitos SRE & Arquitetura	http://localhost:8080/dashboard#concepts	Hub pedagógico de baixo nível (cgroups, CFS, epoll, probes)
Monitoramento de Error Budget	http://localhost:8080/api/v1/slo/error-budget	Disponibilidade real em 30d, consumo e burn rate de SLO (v4.0)
Health Check da Aplicação	http://localhost:8080/healthz	Health check com cache TTL 5s e latência real do banco
Métricas Prometheus (RED)	http://localhost:8080/metrics	Métricas de Rate, Errors e Duration prontas para scraping

Opção 2: Modo Docker Compose (Multi-Network & Tracing) [STACK SEGURA]

Sobe o ecossistema completo em contêineres: Nginx (Proxy reverso porta 80), FastAPI (API v4.0), PostgreSQL 15 com pgvector, Redis e Jaeger Tracing interligados pela rede isolada backend_net.

Passo 1: Execute o build e inicialize os serviços em background:

```
cd /home/magno/sre_rag_agent
docker compose up -d --build
# Ou use o atalho homologado: make compose-up
```

Passo 2: Verifique a saúde e o status de inicialização dos contêineres:

```
docker compose ps
```

Passo 3: Acesso aos serviços via Nginx e Jaeger Tracing:

Interface Web Principal	http://localhost/dashboard	Porta 80 padrão com GZip e SSE ativo
Painel Jaeger Tracing	http://localhost:16686	Visualização de spans OTel distribuídos (v4.0)
Métricas Prometheus	http://localhost/metrics	Coleta RED pronta para scrape
Health Check Nginx/API	http://localhost/healthz	Status com cache defensivo de 5s

Passo 4: Encerrar a stack quando finalizar os testes:

```
docker compose down
# Ou use o atalho homologado: make compose-down
```

Opção 3: Modo Cluster Kubernetes (Kind + Helm v4.0) [PRODUÇÃO REAL]

Simulação completa de cluster K8s local com orquestração Helm (Chart v0.6.0, appVersion 4.0), PSS Restricted, Zero-Trust NetworkPolicies, Prometheus Operator, Alertmanager e Grafana.

Passo 1: Provisionar o cluster Kind e a stack de monitoramento:

```
cd /home/magno/sre_rag_agent
make setup
```

Passo 2: Realizar o deploy do SRE RAG Agent via Helm (v4.0):

```
make deploy
```

Passo 3: Iniciar o encaminhamento de portas (Port-Forward):

```
make port-forward
```

Passo 4: Painéis e Credenciais de Acesso ao Cluster:

Aplicação SRE RAG	http://localhost:8080/dashboard	Sem autenticação (Ambiente de Lab)
Prometheus	http://localhost:9090	Métricas PromQL, SLIs e Recording Rules
Grafana	http://localhost:3000	**admin** / **admin** (Dashboards SRE)
Alertmanager	http://localhost:9093	Alertas ativos, rotas e inibições

Passo 5: Desligar e limpar todos os recursos do cluster:

```
make clean
```

Validação Rápida da Suíte de Testes & Qualidade (v4.0) [QUALIDADE & ACEITE]

Rotina completa de validação contínua para certificar que o Agente SRE OODA, a busca híbrida 3-stage com pgvector, o orquestrador de chaos real no kernel e os 15 critérios de aceite da v4.0 estão 100% íntegros e operacionais.

Passo 1: Script de Validação Automatizada da v4.0 (15 Critérios de Aceite):

```
bash scripts/verify_v4.0.sh
# Valida os 15 critérios: pgvector, flashrank, appVersion 4.0, sre_agent.py, tools.py,
# chaos_real.py, telemetry.py, PSS Restricted, NetworkPolicies, E2E e desafios.
```

Passo 2: Executar a suíte de testes unitários e de integração (17 testes):

```
make test
# Executa: PYTHONPATH=app ./app/.venv/bin/pytest tests/ -v (17/17 PASSED)
```

Módulos e Escopo de Testes Automatizados Cobertos:

Módulo de Teste	Arquivo	O que é validado
Agente OODA & Chaos Real	tests/e2e/test_v4_autonomous_flows.py	Ciclo OODA nos 4 estágios, injeção de cgroup OOM real, Dead Man's Switch 60s, cálculo dinâmico de Error Budget e busca híbrida RRF.
Dashboard & Streaming SSE	tests/e2e/test_dashboard_e2e.py	Renderização do dashboard v4.0, SSE broadcaster pub/sub em tempo real, ciclo de 16 cenários e toggle do Chaos Monkey.
Cenários de Incidente	tests/test_incidents.py	16 cenários de produção, integridade dos runbooks em 5 fases, balanceamento em 4 dificuldades e histórico de zero-repetição.
Chaos Monkey 2.0	tests/test_chaos.py	Injeção controlada de falhas HTTP 500, auditoria em tempo real, geração de eventos de incidentes e exportação de métricas RED.
Motor RAG Semântico	tests/test_rag.py	Indexação vetorial de runbooks, inferência com SentenceTransformer (asyncio.to_thread), similaridade e acurácia de recuperação.

Passo 3: Executar a avaliação de acurácia com o benchmark SRE-Eval:

```
make eval
# Executa: PYTHONPATH=.:app ./app/.venv/bin/python tests/eval/evaluator.py
# Gera o relatório de acurácia, relevância semântica e latência de resposta da IA.
```

■ Dicas de Observabilidade & Troubleshooting Rápido no Lab:

- **Status dos SLOs em Tempo Real:** Consulte /api/v1/slo/error-budget para verificar a taxa de queima instantânea.
- **Rastreamento Distribuído no Jaeger:** Acesse http://localhost:16686 para inspecionar os spans de cada requisição e log.
- **Chaos Real com Dead Man's Switch:** Teste falhas de kernel em /api/v1/chaos/real/cgroup-oom com reversão segura em 60s.
- **Auditoria DevSecOps com Trivy:** Execute trivy fs . para garantir conformidade contínua e 0 CVEs críticas.